

7-3 · Embedding과 Vector DB · 실습 1H

임베딩 생성 실습

차원을 세고, 유사도를 손으로 계산한다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① langchain-openai(또는 ollama)와 qdrant-client 를 설치한다
- ② embed_query 로 벡터를 만들고 차원(1536 또는 768)을 확인한다
- ③ cos 함수를 짜서 비슷한 문장과 다른 문장의 값을 비교한다
- ④ 문장 5개의 유사도 표를 만들고 배치 처리 속도를 쟀다

여기서 메모한 차원 수를 다음 1.5H 실습의 컬렉션 size 에 그대로 씁니다

여섯 단계 — ~/data-eng/week7 의 .venv 안에서

OpenAI 키와 로컬 Ollama 중 하나를 고릅니다 — 둘 다 없어도 다음 실습은 가짜 벡터로 진행할 수 있습니다

1. 설치	2. 차원 확인	3. 유사도	4. 유사도 표	5.6. 배치·한계
langchain-openai qdrant-client	embed_query len(v1)	cos 함수 직접 계산	문장 5개 5×5 표	embed_documents 부정문·식별자

1단계

패키지 설치와 확인

가상환경	cd ~/data-eng/week7 · source .venv/bin/activate
선택 A — OpenAI	pip install langchain-openai
선택 B — Ollama	pip install langchain-ollama · ollama pull nomic-embed-text
공통	pip install qdrant-client
확인	pip list grep -i "openai\ ollama\ qdrant"

API 키는 코드에 적지 않는다 — 7-1에서 만든 .env 에 두고 환경변수로 읽는다 · .env 가 .gitignore 에 있는지 다시 확인

임베딩을 만들고 차원을 확인

선택 A — OpenAI (1536차원)

text-embedding-3-small 모델

- from langchain_openai import ...
- OpenAIEmbeddings(model=...)
- len(v1) → 1536

VS

선택 B — Ollama (768차원)

nomic-embed-text 모델 · 로컬

- from langchain_ollama import ...
- OllamaEmbeddings(model=...)
- len(v1) → 768

확인 포인트 — 차원 수, 그리고 유사도 값

1

차원이 1536(또는 768)

값은 -1~1 근처의 소수 — 이 숫자를 메모, 다음 실습의 size 에 쓴다

2

v2 「배치 작업을 하루에 한 번 돌립니다」

v1 「데이터 파이프라인을 매일 실행합니다」와 겹치는 단어가 거의 없다

3

v3 「오늘 점심은 김치찌개입니다」

「비슷한 문장」 값이 「다른 문장」보다 뚜렷하게 높아야 한다

키워드 검색이라면

0점

$\cos(a, b) = \text{dot} / (na \times nb)$ — 이론에서 본 네 줄을 그대로 짠다

문장 5개의 유사도 표 — 예상과 대조

번호	문장	예상
0	ETL은 추출, 변환, 적재 순서로...	1과 가장 가깝다
1	ELT는 먼저 적재하고 나중에 변환	0과 가장 가깝다
2	Airflow는 작업 순서를 관리하는 도구	—
3	Kafka는 이벤트를 실시간으로...	—
4	오늘 날씨가 참 좋습니다	나머지 전부와 가장 멀다

대각선은 1.00(자기 자신) · 예상과 다른 쌍이 있으면 왜 그런지 이야기해 보세요 — 임베딩의 한계를 아는 기회

하나씩 vs 한꺼번에 — time 으로 재기

하나씩 — embed_query

for 문으로 문장마다 한 번씩 호출



VS

배치 — embed_documents

리스트를 통째로 한 번에 호출



임베딩이 약한 곳 — 부정문과 식별자

- 1 「성공했습니다」 vs 「실패했습니다」**
반대말인데도 유사도가 높게 나온다 — 문맥이 거의 같기 때문
- 2 「주문번호 A-1001」 vs 「A-1002」**
숫자·코드는 잘 구분하지 못한다
- 3 그래서**
정확한 식별자를 찾는 일은 벡터 검색에 맡기지 않는다 — 키워드·조건 검색으로
- 4 다음 실습에서**
payload 필터(조건 검색)와 의미 검색을 함께 쓰는 방식을 본다

이론의 비교표를 기억하세요 — 정확한 코드·품번은 키워드 검색이 강하다

이 1시간이 끝날 때 확인할 것

- 1 pip list 에 openai(또는 ollama)와 qdrant 가 보인다
- 2 len(v1) 이 1536 또는 768 — 메모해 두었다
- 3 「비슷한 문장」 값이 「다른 문장」보다 뚜렷하게 높다
- 4 유사도 표의 대각선이 1.00, 날씨 문장(4)이 가장 멀다
- 5 배치(embed_documents)가 하나씩보다 훨씬 빠르다
- 6 성공 vs 실패, A-1001 vs A-1002 의 유사도가 높게 나왔다

7-3 · 실습 1H 마무리

다음은 Qdrant 실습 (1.5H)

Qdrant 를 도커로 띄우고 컬렉션을 만들어 오늘 만든 벡터를 적재하고, 단어가 겹치지 않는 질문으로 검색합니다

✓ **docker run** 으로 qdrant 기동

✓ **size·distance** 로 컬렉션 생성

✓ 차원을 일부러 틀려 오류 보기

✓ **query_points** 로 의미 검색·필터

가져갈 것

emb 객체와 **v1** · 메모한 차원 수